RecRec: Recursive Refinement for Sequential Recommendation

Anonymized for Review

Appendix

This document provides ablation experimental results, detailed implementation specifications, and extended computational analysis for RecRec.

A. Implementation Details

We implement RecRec in PyTorch, fixing embedding sizes to 384 for all baselines and using a maximum sequence length of 50. We used outer recursion steps (T=7) and inner recursion loops(n=3) in our experiments. Following the leave-one-out protocol, we pair each ground-truth item with 99 negative samples to form a 100-item candidate set. Models are trained for up to 50 epochs with a batch size of 512 using Cross-Entropy loss and the Adam optimizer ($lr = 10^{-3}$). To enhance stability, we apply exponential moving average (EMA) with a decay rate of 0.999. All experiments were conducted on a single NVIDIA RTX 3090 GPU.

B. Detailed Ablation Study

To investigate the contribution of each component in our architecture, in addition to steam games in main paper, we conduct an ablation study across the Luxury Beauty and Video Games datasets. We evaluate three variants: (1) **No correction gate (G)**, (2) **No inner recursion (R)** and (3) **No latent** Z.

Table 1: Ablation Study on Luxury Beauty and Video Games Datasets

Configuration	**Luxury Beauty**									**Video Games**								
	HR@K			**NDCG@K**			**Prec@K**			**HR@K**			**NDCG@K**			**Prec@K**		
	@1	@5	@10	@1	@5	@10	@1	@5	@10	@1	@5	@10	@1	@5	@10	@1	@5	@10
No G	0.33	0.42	0.51	0.33	0.38	0.39	0.33	0.09	0.05	0.30	0.50	0.59	0.30	0.41	0.48	0.30	0.09	0.07
No R	0.39	0.53	0.61	0.39	0.45	0.45	0.39	0.10	0.06	0.33	0.56	0.63	0.33	0.46	0.52	0.33	**0.12**	0.07
No Z	0.36	0.50	0.57	0.36	0.40	0.41	0.36	0.10	0.06	0.29	0.47	0.52	0.29	0.38	0.44	0.29	0.09	0.07
Full	**0.43**	**0.59**	**0.66**	**0.43**	**0.51**	**0.53**	**0.43**	**0.12**	**0.06**	**0.36**	**0.61**	**0.71**	**0.36**	**0.49**	**0.52**	**0.36**	**0.12**	0.07

C. Analysis of Model Confidence

We examine the model's *Confidence Margin* (M), defined as the logit distance between the target item and its most competitive "distractor" in the candidate set. For a given refinement step t, let

y_t be the predicted preference vector and $\mathbf{e}_i$ be the embedding of candidate item i. The score is computed as $s_{t,i} = y_t \cdot \mathbf{e}_i^\top$. The confidence margin M_t is formally defined as:

$$M_t = s_{t,\text{target}} - \max_{j \neq \text{target}} s_{t,j} \tag{1}$$

where $s_{t,\text{target}}$ is the logit of the ground-truth item. As shown in Figure 1, the margin peaks at $T = 7$.

Table 2: Impact of recursion steps on HR@1 (Steam dataset)

Recursion Steps	**HR@1**
$T = 1$	0.49
$T = 3$	0.57
$T = 7$	0.59
$T = 13$	0.55

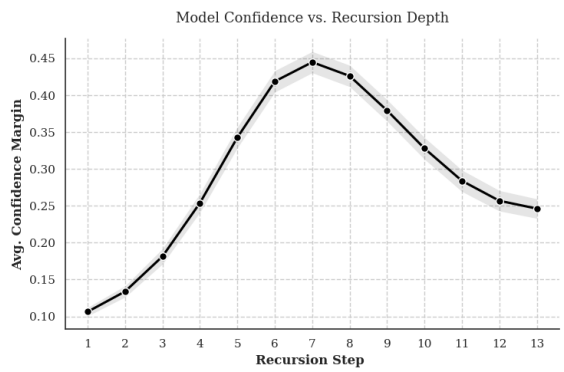

Figure 1: Step-wise confidence curve

D. Computational Efficiency

Table 3 presents the latency for traditional baselines, LLM-based recommenders, and our proposed model. RecRec completes training in approximately 210 minutes on the Video Games dataset, representing a reduction in training time of over 98% compared to LLM-based recommenders.

Table 3: Run Time Comparison (Minutes)

Model	**Luxury Beauty**	**Video Games**	**Steam**
SasRec	40	180	50
NGCF	25	75	40
LightGCN	35	80	60
UniRec	100	250	180
ReaRec	90	290	170
Allmrec	3,700	12,400	5,600
Tallrec	6,700	19,800	10,300
Gpt4rec	1,200	5,200	2,300
RecRec (Ours)	**55**	**210**	**90**